# Technical Interview Questions - Part 4 - Git + SQL vs noSQL

Giuliana Olmos · 14 de noviembre de 2024 · https://dev.to/giulianaolmos/technical-interview-questions-part-4-git-sql-vs-nosql-28l1

## Introduction

Hello!! I hope you're doing well. 🫶🏻

In this post, I'll talk about two different topics, because I didn't want to write a post with just ONE question. These aren't common interview questions, but I had to answer them in two separate situations. Since they were so unusual, I didn't respond as well as I could have.

So, I want to help you by sharing some points that, in my case, the interviewers expected me to mention.

## Questions

1. Difference between Git Merge and Git Rebase
2. What are the differences between SQL and NoSQL databases?

---

### Question 1: Difference between Git Merge and Git Rebase

*The short answer:*

> Both **Git rebase** and **Git merge** are ways to integrate changes from one Git branch into another, but they work differently.

> - **Git rebase**: Rewrites the commit history of the original branch.
> - **Git merge**: Creates a new commit that combines both branches.

...

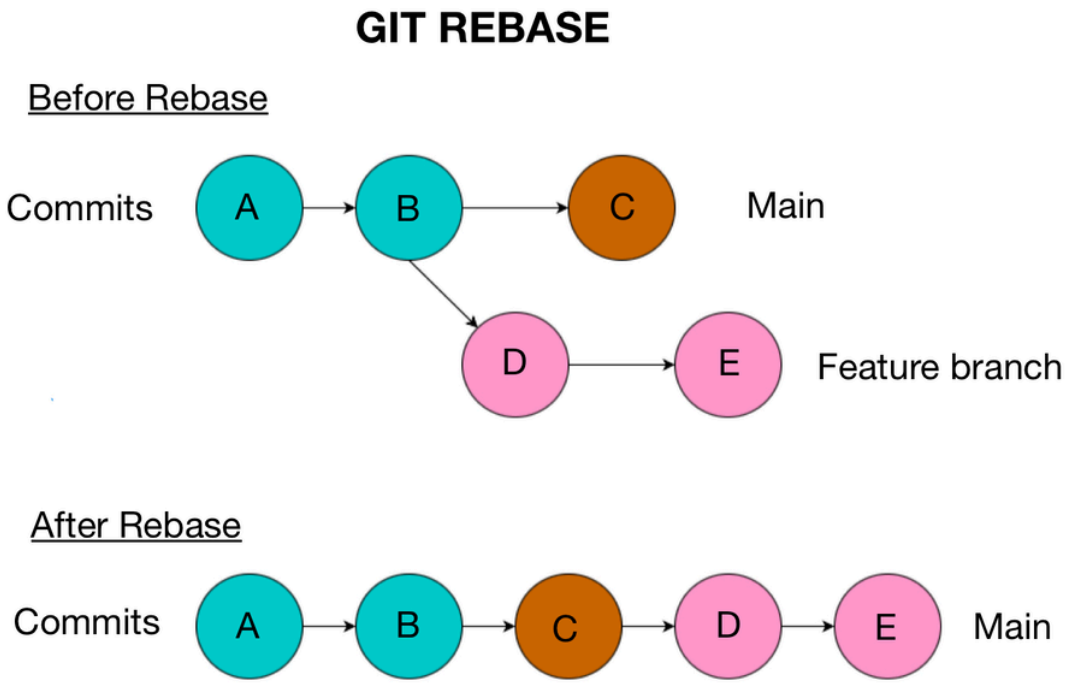
#### GIT REBASE

**Git rebase** integrates changes from one branch into a new base branch by replaying all the commits from the old branch onto the new one.

This command rewrites the commit history of the original branch and creates a new branch by applying the same set of commits on top of the target branch.

**Syntax:**

```
git rebase <base>
```




**When should you use it?**

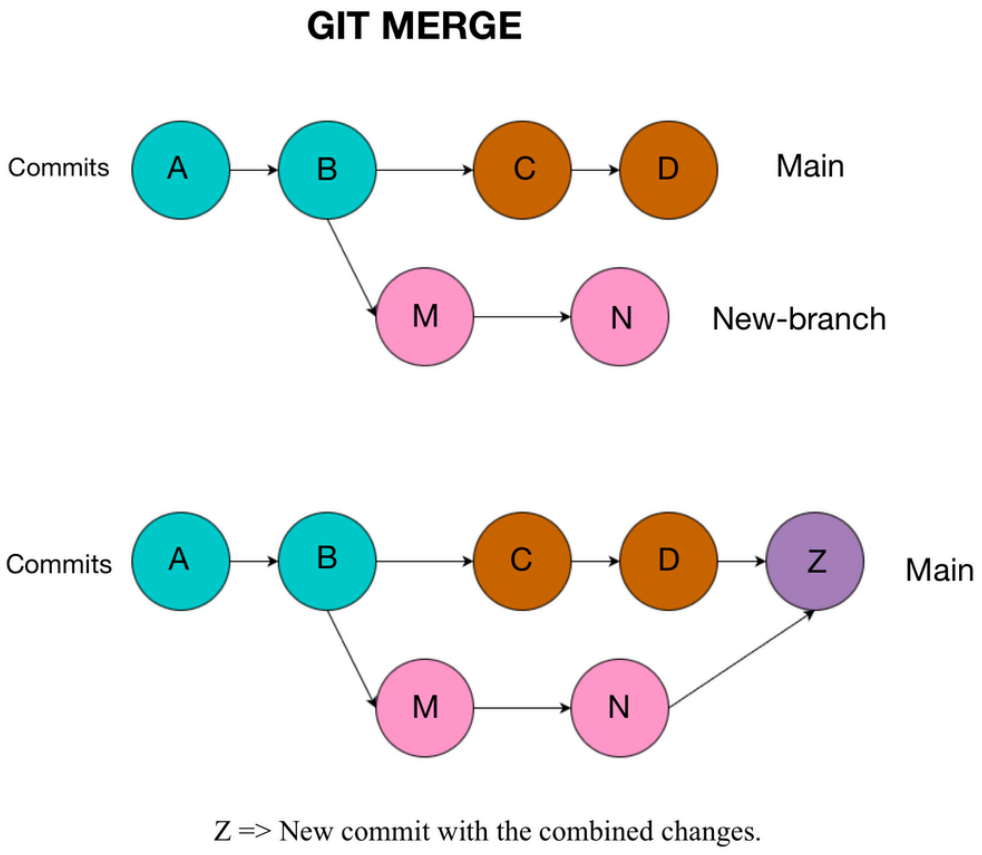
- When you want to incorporate changes from a feature branch into the main branch.
- When you don’t need to track individual commits from the feature branch (because all commits will be rewritten and added on top of the target branch).

...

**GIT MERGE**

**Git merge** allows you to merge changes from two different branches into one, usually the main branch. In this process, Git creates a new commit to combine the changes.

```
git merge <branch>
```




Z => New commit with the combined changes.

**When should you use it?**

- When you want to incorporate changes from one branch into another without losing the commit history.
- When you know you'll need to check your past commits.
- When you're working with another developer and want to keep the full commit history intact.

---

## Question 2: What are the differences between SQL and NoSQL databases?

*Answer:*

> - **SQL** uses structured data, while **noSQL** does not.
> - **SQL** scales vertically, whereas **noSQL** scales horizontally.
> - In some situations, **noSQL** can be faster than **SQL**.

I want to dive more into these points.

**SQL** databases contain structured data and relationships that are handled using variables and keys. On the other hand, **noSQL** databases handle unstructured data with dynamic schemas, and they are not designed to handle complex relationships.

**Scalability**

- **SQL** scales better vertically (Scale Up). This means that to handle more data or higher workloads, you need to increase the computational power of a single server (e.g., by adding more CPU, RAM, or storage).
- **noSQL**, however, scales better horizontally (Scale Out). Instead of upgrading a single machine, you add more servers to handle increasing loads.







***Is noSQL faster than SQL?*** In some cases, **noSQL** can be faster than **SQL** because it is schema-less and doesn't need to follow ACID properties. Since there's no need to validate schemas, data writing and retrieval can be faster. Additionally, **noSQL** often uses denormalized data, meaning there is less need for **JOIN** operations, which are among the most performance-heavy operations in **SQL**.

However, **SQL** is generally more efficient when working with:

- transactional systems,
- complex queries, and
- situations where strict data consistency and structure are required.

---

## Farewell & Thoughts

I hope these explanations help you in your interviews.
If you have any interview questions you'd like me to answer, or if you want to share your experience, I'd love to hear about it.

See you in the next post!